

Schema-Driven Export Restructure

OpenGOAL Level Tools — Jak 1 Level Builder

Branch `feature/schema-driven-export` · overview / handoff

The idea

Each actor in the game database carries a declarative **fields[]** schema that describes its settings, value types, defaults, and the export lumps it writes. That one schema is now the single source of truth for **both** the Blender settings UI and the OpenGOAL exporter — an actor is described once, and both sides read the same description instead of maintaining parallel logic.

Why

The trigger was a concrete bug: a sync platform placed with **no waypoints** silently lost its **sync** lump, because the exporter only wrote **sync** inside a “has waypoints” branch. The deeper issue was structural — export logic lived in scattered, hardcoded per-actor branches, kept separately from the UI that configured those same actors. The two could drift apart, and edge cases like a pathless platform fell through the gap. Describing an actor once removes that entire class of mismatch.

What changed

- A small, pure export **engine** reads an actor's **fields[]** and produces its lumps directly: value types, default handling, conditional emission, enums, and bit-flags.
- **36 actors** now export from their schema instead of an old hardcoded branch. For every one, the schema output was verified **identical** to the previous output — so existing levels export unchanged — with one deliberate exception: the pathless-sync bug is now fixed.
- The Blender settings panel became **generic**: it renders any actor's **fields[]** automatically. Most actors now share one panel, and a newly added actor gets its settings UI with no extra code.
- Genuinely **computed** lumps — entity links, geometry-derived values, picker logic — stay in code on purpose. The schema yields to them, so nothing computed is lost.

Where

File	Role
<code>jak1_game_database.jsonc</code>	The schema itself — each actor's fields[] and its <code>schema_export</code> flag
<code>export/schema_emit.py</code>	The engine — turns fields[] into lumps; pure, no Blender dependency
<code>export/actors.py</code>	The hook — applies the schema during export, yielding to computed & custom lumps
<code>db.py</code>	Schema inheritance and the <code>schema_export</code> enable check
<code>panels/actor_fields.py</code>	The generic, schema-driven settings panel
<code>export/test_schema_emit.py</code>	Tests pinning the schema conventions to expected output

How this moves toward the goals

- **Authoring becomes data, not code** — adding or tuning an actor is a database edit, not a code change.
- **UI and export can no longer drift** — they read the same schema by construction.
- **The original bug class is eliminated** for migrated actors: dropped or gated lumps and UI–export mismatch.
- **Incremental and safe** — 36 actors migrated, the rest untouched, computed cases preserved; a clean path to fold in the remainder over time.
- **Validated before any merge** — full-addon compile, schema lint, engine robustness, a lost/added-lump regression diff (zero regressions), a real Blender 4.4.3 load, a byte-identical export of an existing map, and in-game confirmation of the sync fix.

Status. On branch `feature/schema-driven-export`. Held for further testing before merging to `main`; `main` remains a clean, installable build.